

LECTURE 17

Midterm Review

A checkpoint across Lectures 1–16

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

Four Units, One Exam

- 1 Unit 1: Foundations — DBMS, database approach, three-schema architecture
- 2 Unit 2: The Relational Model — keys and integrity constraints
- 3 Unit 3: Relational Algebra & Calculus — operators, TRC/DRC
- 4 Unit 4: ER & EER Modeling — conceptual design and mapping to relations
- 5 Self-test: eight questions in the style the exam will ask

Lectures 1–16, Four Units

Each unit hands the next one a vocabulary it assumes fluency with — and Unit 4 is exactly where normalization (Lecture 18 onward) picks up.

Unit 1: Foundations (L1–4)

DBMS, database approach, DDL/DML, three-schema architecture, data independence



Unit 2: Relational Model (L5–6)

Relations, attributes, domains, keys, integrity constraints



Unit 3: Algebra & Calculus (L7–10)

SELECT/PROJECT, joins, division, aggregation, TRC/DRC

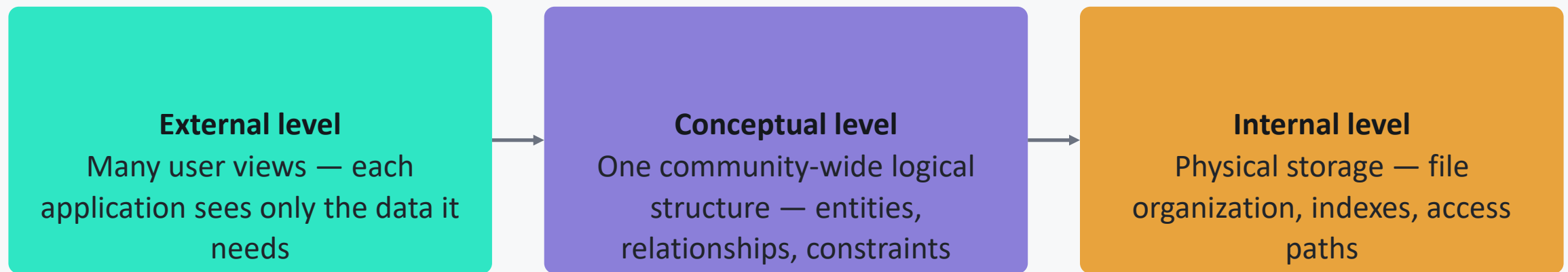


Unit 4: ER & EER Modeling (L11–16)

Conceptual design, ER, EER, mapping to relational schemas

Foundations of Database Systems

- File-based approach: data duplicated across programs, formats tightly coupled to whichever application wrote them
- Database approach: centralizes data and metadata behind one DBMS, shared via DDL/DML
- Payoff: data independence — applications keep working even when storage details change



External–conceptual mapping → logical data independence. **Conceptual–internal mapping** → physical data independence.

The Relational Model: Keys & Constraints

A relation is a named set of tuples over fixed attributes — a set, so no duplicate tuples and every value must be atomic. The key vocabulary nests strictly inside itself:

Term	Definition
Superkey	Any attribute set that uniquely identifies a tuple — may carry extra attributes
Candidate key	A minimal superkey — remove any one attribute and uniqueness breaks
Primary key	The candidate key the designer chooses to identify tuples
Alternate key	Every candidate key that was NOT chosen as primary
Foreign key	An attribute set reproducing another relation's primary key

Integrity constraints keep a schema honest: domain constraint (one value vs. its domain) · entity integrity (no NULL in a PK) · referential integrity (FK matches an existing key, enforced via RESTRICT / CASCADE / SET NULL) · general constraints (business rules)

Relational Algebra: Operator Families

Relational algebra is **procedural** — every operator's output is itself a relation (closure property), so operators nest freely:

```
π staffNo, name (σ salary>20000 (Staff))
```

Family	Operators	Lecture
Unary	SELECT (σ) filters rows, PROJECT (π) filters columns	7
Set	UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE ($-$), CARTESIAN PRODUCT ($\times$)	7
Join / Division	Theta-join, equijoin, natural join ($\bowtie$), outer joins, DIVISION ($\div$)	8
Aggregation	SUM, COUNT, AVG, ... with GROUP BY-style grouping	9

Relational calculus (Lecture 10) answers the same questions declaratively — you state the properties the result must satisfy (TRC uses tuple variables, DRC uses domain variables) and leave “how” to the DBMS. Both are equivalent in power (Codd's theorem), restricted to safe expressions.

Relational Algebra vs. Relational Calculus

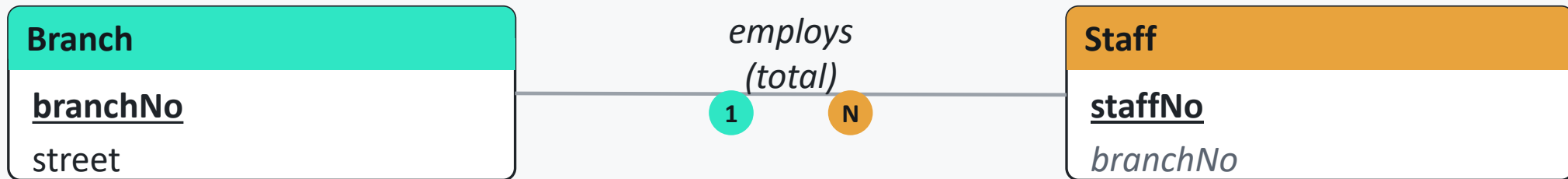
	Relational Algebra	Relational Calculus
Paradigm	Procedural — specify HOW	Declarative — specify WHAT
Notation	$\sigma, \pi, \bowtie, \div, \cup, \cap, -$	$\exists, \forall, \wedge, \vee, \neg$ (TRC/DRC)
Variables	None — operates on named relations	Tuple (TRC) or domain (DRC) variables
Best when...	Tracing a query step by step	Stating a condition without an order
Risk	None — always safe	Unrestricted $\neg$ can be unsafe (infinite result)
Division ($\div$)	A dedicated operator	Nested $\forall/\exists$ quantification — harder by hand

TIP

Write the algebra you trust first, then translate operator by operator into TRC.

ER & EER Modeling

- ER model: entity types, relationship types, attributes, structural constraints (cardinality + participation)
- EER adds: superclass/subclass hierarchies, specialization, generalization, aggregation, composition, categorization
- Lecture 16 is the rulebook mapping every EER construct down to relations



A 1/N badge pair is the cardinality ratio; whether every instance must participate (total) or some may not (partial) is the participation constraint — independent of each other.

ER / EER Constraint Cheat-Sheet

Term	Meaning	Worked example
Cardinality ratio	Max relationship instances one entity can participate in: 1:1, 1:N, M:N	One Branch employs many Staff → 1:N
Total participation	EVERY instance must participate	Every Staff row has a branchNo
Partial participation	SOME instances may not participate	Not every Client has booked a Viewing
Disjoint (d)	An instance belongs to at most ONE subclass	Staff is Manager OR Assistant, never both
Overlapping (o)	An instance MAY belong to several subclasses	A Person could be Student AND Employee
Total specialization	Every superclass instance belongs to ≥ 1 subclass	Every Staff is Manager/Supervisor/Assistant
Partial specialization	Some instances belong to NO subclass	A Vehicle may have no Car/Truck subtype yet

Mapping Specialization to Relations

Option A — one relation, type attribute

Staff(staffNo,...,staffType,bonus,trainingLevel); NULLs. Best: disjoint + total.



Option B — one relation per subclass only

Manager(staffNo,...,bonus), Assistant(...). No Staff table. Best: disjoint + total only.



Option C — superclass + one relation per subclass

Staff + Manager(staffNo,bonus) + Assistant(staffNo,...). Best: overlapping or partial.

WARNING

Option B breaks under partial specialization; Option C always works.

When to Use Algebra vs. Calculus

- Reach for algebra when: you're asked for a step-by-step derivation, need to reason about intermediate relations, or translate to an execution plan
- Reach for calculus when: you're asked to state a condition concisely without committing to an order of operations
- Both are equally correct for anything expressible safely in either (Codd's theorem) — “which is right” is rarely the question; “which is asked for” is

“Find every client such that no property they'd accept is currently vacant”
reads far more naturally as a quantified calculus formula than a nested algebra expression.

Algebra & Calculus

q1. Write the relational algebra expression: names of staff at branch B003 earning more than 15,000.

$\pi \text{ name } (\sigma \text{ branchNo}='B003' \wedge \text{salary}>15000 (\text{Staff}))$ — SELECT (σ) runs first while branchNo/salary are still present, then PROJECT (π) narrows to name.

q2. Write the equivalent tuple relational calculus (TRC) expression.

$\{ S.\text{name} \mid \text{Staff}(S) \wedge S.\text{branchNo}='B003' \wedge S.\text{salary}>15000 \}$ — “the set of name values for every tuple S in Staff such that S's branch is B003 and salary exceeds 15000.”

Keys & Referential Integrity

q3. Difference between a candidate key and the primary key of a relation?

A relation may have several candidate keys — minimal sets that each independently guarantee uniqueness. The primary key is the ONE candidate key the designer chooses; every other candidate key becomes an alternate key. No structural difference until that choice is made.

q4. Branch B003 is deleted while 3 PropertyForRent rows still reference it. RESTRICT vs. CASCADE?

RESTRICT: the delete is refused as long as any PropertyForRent row references B003 — properties must be reassigned/deleted first. CASCADE: the delete succeeds and automatically removes all 3 referencing rows too.

Division and Safe Expressions

Q6. What question does DIVISION ($\div$) answer that SELECT/JOIN struggles to express?

Division answers "for every" questions, e.g. find every client who has viewed every property managed by branch B003. It returns exactly the clients whose viewed set contains the whole divisor set.

Q7. Why must relational calculus expressions be safe?

An unrestricted formula (e.g. heavy use of $\neg$ over an unbounded domain) can describe an infinite result. Safety restricts formulas to values that actually appear in the database.

EER Specialization

q5. Staff specialization: Manager (bonus), Assistant (trainingLevel), disjoint + total. Using Option C, write the schemas.

Staff(staffNo, name, position, salary, branchNo)

Manager(staffNo, bonus) — staffNo is PK and FK referencing Staff

Assistant(staffNo, trainingLevel) — same pattern. (Option B would also work here, but C is the safe general-purpose choice.)

q8. Distinguish disjoint vs. overlapping, and total vs. partial specialization, in one sentence each.

Disjoint: an instance belongs to at most one subclass. Overlapping: it may belong to several at once. Total: every superclass instance must belong to some subclass. Partial: some instances may belong to none.

KEY TAKEAWAYS

Ready for the Midterm

- 1 Units 1–4 build strictly on each other: DBMS/architecture → relational model → algebra/calculus → ER/EER design
- 2 Three-schema architecture is the mechanism behind data independence — know which mapping buys which kind
- 3 Key hierarchy nests: candidate key $\supseteq$ primary key; superkey $\supseteq$ candidate key (minimality separates them)
- 4 Algebra is procedural (how); calculus is declarative (what) — both equally expressive over safe expressions
- 5 Mapping EER to relations comes down to 3 specialization strategies — Option C never breaks
- 6 Lecture 18 onward (normalization) assumes every idea here is second nature — especially FDs and keys